Declared but Not Sampled: A Benchmark Defect, and Nine Ways Our Verification Missed Our Own Errors

Anonymous submission

Abstract

This paper reports a benchmark defect, a criterion for when such defects matter, and — from the same work — nine occasions on which our own verification passed while our claim was false. The third is the part we expect to outlive the other two.

The defect. Every LIBERO task declares, in its own task file, a region from which to sample its target's starting position. Three of the four suites ship evaluation states that exercise that region. **LIBERO-Object does not**: it declares the largest region of the four, $5 \times 5\,\text{cm}$, and ships the smallest spread — on six of its ten tasks all fifty released states place the target at a single point, to the last bit of a `float64`. This is arithmetic on two shipped files. We ship the repair, validate it, and show it bites: drawing the fifty states from the region the task file itself declares, and changing nothing else, costs a lookup policy half its score ($25/30 \to 10/30$, paired, exact McNemar $p = 0.0007$), while a controller handed the true target once at reset succeeds on *all thirty* of those same repaired states. The states are not harder; the constant is wrong on them. The defect is exploitable, not cosmetic.

The criterion. A frozen placement matters only if the robot cannot resolve the freezing. Define the *placement radius* R of an object as the radius of the smallest disc containing every position the benchmark ships for it. A single task-indexed constant serves that object *if and only if* $R \leq \delta$, the tolerance below which the embodiment cannot tell two placements apart. On the object its tasks are *named* by, LIBERO-Object satisfies this on all ten tasks at the tolerance we measure, against two of thirty elsewhere. We report in full how weakly δ is determined — it is the weakest link, and our own two estimates of it disagree.

What verification did not catch. We built the lookup policy the criterion describes. It settles the manipulation check above and not the comparison we wanted — whether a constant matches a trained policy — and we say why. Along the way we made nine errors, each found by adversarial review rather than by us, and they share a structure. **Not one was a wrong computation.** Each was a decision about which slice of a correct instrument's output got written down: which trials had finished, which axes were compared, which tolerances were swept, which of two estimates was quoted, which parts of the document a retraction reached. Our verifier recomputes every number in the load-bearing sections from raw outcomes — and it passed, at 76 checks, while the central claim it certified was false, because it recomputed the two-dimensional distance we had chosen to compute. **All nine ran in the direction that flattered us.** We give the taxonomy, the artifacts, and the review process that produced it, because a field that checks arithmetic is defended against none of this.

1 The question

A policy scores 0.700 on a manipulation benchmark — 35 successes in 50 trials on one LIBERO-Object task, and it is our own policy. What has been certified?

The intended reading is a capability claim: the policy perceives the named object, locates it, and manipulates it. The reading this paper is about is narrower and much cheaper: the policy has learned *where the object always is*. Both readings produce the same number. Distinguishing them is not a matter of looking harder at the policy — two policies can be indistinguishable to a probe and opposite in the world, and we exhibit such a pair — but of first measuring the benchmark.

The framing is not new. While this work was in revision, Jiang et al. [5] defined *shortcut solvability*: a benchmark score is evidence of capability only if every policy reaching it has the capability. That is our Corollary 1 in content. They audit five manipulation benchmarks including all of LIBERO and show a 0.09B probe with no language encoder scoring within one point of the best published result on three of LIBERO's four splits, and 100.0% on LIBERO-Object — a stronger demonstration of it than ours.

Our *blind* arm is the manipulation instance of the unimodal ablation Thomason et al. [11] established in embodied AI, and measuring a benchmark's degeneracy before crediting a model is standard outside robotics (Table 1). We claim no novelty for either move, and that VLAs ignore language is not our finding.

	probes from	cannot say
Benchmark degeneracy, outside robotics		
Torralba & Efros [12]	dataset identity	—
HANS [8], artifacts [4]	hypothesis only	where in a stack
ImageNet-v2 [10]	a second test set	—
Shortcuts [3]	general statement	—
Perturbation, in robotics		
LIBERO-PRO [13]	re-running with perturbed scenes	how much was gameable *a priori*
LIBERO-Plus [2]	seven axes, ten VLAs	which stage reads the shortcut
robomimic [7], COLOSSEUM [9]	re-collection, perturbation	what the shipped files already permit
This paper	**shipped files, before training**	**whether the radius binds: that needs δ, and δ needs a policy**

Table 1: Every prior entry needs a trained model and a run to say anything. The placement radius is computed from the benchmark's own initial-state files with no policy, no simulator and no training, which is what lets it bound what a score *could* certify rather than report what one model did.

What is ours is the *a priori* half. Every *prior* diagnostic in Table 1 needs a trained model and a run. R is a property of the shipped files: it bounds what a score could certify before a GPU is switched on, and it names *which* object in a task carries the degeneracy.

It is also not the whole story. Jiang et al.'s probe succeeds on Spatial, Goal and Long, where our own Table 6 shows large radii — so pinning cannot be why. On those splits the route is visual task-identifiability without language, which §7.1 independently supports. The two findings converge rather than compete.

Contributions.

1. **A benchmark defect, measurable from shipped files** (§2). LIBERO-Object's released evaluation states do not sample the region its own task files declare; three sibling suites do. No tolerance, policy or simulator is required to check this, and the fix is already specified by the benchmark.
2. **Evidence that it is exploitable, not cosmetic** (§2.1). Sampling the declared region and changing nothing else costs a lookup policy half its score — $25/30 \to 10/30$, paired, exact McNemar $p = 0.0007$. This is the manipulation check for the defect, and a null result would have retired the criterion below.
3. **A criterion for when such a defect is exploitable** (§3). The placement radius R, with the exact condition $R \leq \delta$ — necessary and sufficient, not merely sufficient — and a measurement of all four LIBERO suites against it.
4. **A behavioural test that fails, with its diagnosis** (§5–§6). We built the lookup policy and it cannot decide the question; we say why, at the correct unit of analysis, and name the experiment that would.
5. **Certification methodology** (§9): a counterexample to probe-only certification, and the demonstration that matching a published rate is not evidence of having reproduced a sample.
6. **Nine errors, and what they share** (§11). Every prediction we registered including the failures, every instrument retired at its calibration gate, and nine errors of our own — one found inside the correction of another. **None was a wrong computation**; each was a choice about which slice of a correct instrument's output got reported, and all nine ran in the direction that flattered us. Our own verifier passed while the claim it certified was false.

What is regenerable, and what is not. `scripts/verify_paper_numbers.py` recomputes 129 values across §2–§6 from the per-trial outcomes and the shipped initial states — including the tests those sections turn on — and fails on disagreement. A second script, `scripts/check_manuscript_consistency.py`, checks the manuscript against itself, because two of our nine errors were corrections that reached the section that made a claim and not the sections that cite it.

An earlier version of that script checked p-values by reading them back out of the JSON that wrote them — a consistency check between two copies of a number, not a re-derivation. A reviewer caught it; it is fixed. The script names the claims it does *not* cover, and one of them matters: the historical $0.700 = 35/50$ cell that opens this paper was measured before we shipped per-trial logs, and its trial record is not in the repository.

Figure 1 is the map: it tiers every claim below by what it rests on, so a reader can see at once which survive if the tolerance is disputed.

Scope, stated once and up front. All results are simulation. Every confirmatory cell is one task and one object unless stated. Cell sizes are given with every number and no cell is quoted without its interval. Our external validation attempt is reported as a *negative* (§10): we could not reproduce a public checkpoint's published performance on our build, and we do not report probe results on a harness whose baseline we cannot reproduce.

2 A benchmark that does not sample what it declares

We begin with the fact that needs the least machinery. It compares two files the benchmark ships. No tolerance, no controller, no policy, no simulator, and no statistics enter it.

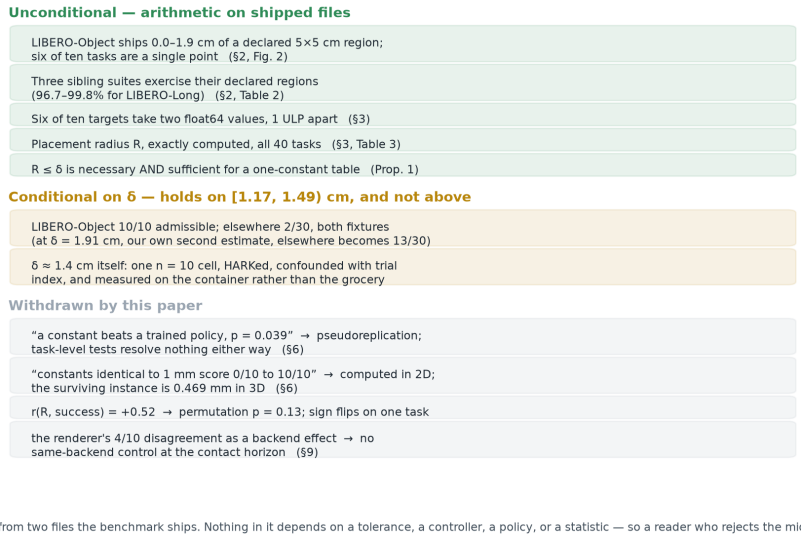


Figure 1: **What each claim in this paper rests on.** The tolerance δ is this work's weakest link — our two measurements of it disagree by a factor of 1.4, and the separation it licenses lives on a 0.32 cm window — so rather than leave a reader to reconstruct the dependency from prose, we draw it. **Nothing in the top tier depends on δ, on a controller, on a policy, or on a statistic**: it is arithmetic on files the benchmark ships. A reader who rejects the middle tier outright still keeps the top one. The bottom tier is claims this paper made and withdrew, kept visible so the record is legible. Generated by `paper/render_fig12_dependency.py`.

Each LIBERO task declares, in its own BDDL, an axis-aligned region to sample its target from. For `pick_up_the_alphabet_soup` that region is 5×5 cm. The fifty shipped initial states place the soup at a single point — the box's exact centre (Figure 3). **The generator randomises; the released evaluation states do not.** Figure 2 shows this is not one chosen task: it draws all twenty tasks of the two suites that declare the same box.

suite	declared	shipped spread	exercised?
Spatial	2.0 cm	2.9 cm	yes
Goal	2.0 cm	3.0 cm	yes
Long	5.0 cm	4.9 cm	yes
Object	**5.0** cm	**0.0–1.9** cm	**no**

Table 2: **Three LIBERO suites ship states that exercise the region their task files declare. One does not.** Mean declared box side against mean shipped spread. LIBERO's sampler places within the declared box plus a margin, so a suite doing its job ships a spread at or slightly *above* its declaration — Spatial, Goal and Long all do. LIBERO-Object declares the largest box of the four and ships the smallest spread, 0.0 cm on six of ten tasks, where all fifty states are one point. Long is the sharpest control: it declares the *identical* 5 cm box and ships 96.7–99.8% of it, so this is not a property of the generator. Arithmetic on two shipped files — no δ, no policy, no simulator. Artifact: `results/declared_vs_shipped.json`.

Table 2 gives all four suites. So LIBERO-Object's pinning is not a design decision to be argued with. It is a **state-generation defect**: the benchmark's own task files specify the fix, and a suite in the same release demonstrates it working.

We ship the repair, not a recommendation. The fix requires no redesign: draw the target's position from the box the task file already declares. `scripts/resample_init_states.py` does this for all ten LIBERO-Object tasks (Table 3), changing *only* the two state-array columns holding the target's x and y — every other object, every joint velocity and the time field are bit-identical to the released file, which we verify by diffing them.

	shipped	repaired
spread, LIBERO-Object (10 tasks)	0.00–1.94 cm	4.44–4.96 cm
spread, LIBERO-Long (unmodified)	4.84–4.99 cm	
clearance to any other region	—	$\geq$ 7.42 cm

Table 3: The repaired LIBERO-Object states span the region their task files declare and land on the sibling suite's spread. Positions are drawn under a clearance constraint, not from the whole box: **the first version of this script sampled the box freely, and validation showed it would have placed objects in contact on five of ten tasks**, because those targets' declared regions come within 5.0 cm of a distractor's while the assets' own collision geoms (footprint radii 2.2–3.7 cm) need 7.42 cm centre-to-centre. `scripts/validate_resampled.py` now checks every written position against every declared region and against the free-body clearances the shipped states exhibit; all ten tasks pass. Two things it cannot settle without a simulator — resting height and table contact after settling — are listed rather than assumed, and the manifest labels the suite a candidate repair.

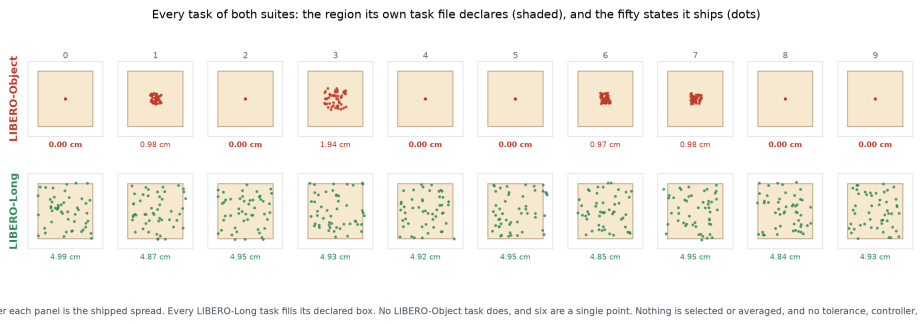


Figure 2: **Every task of both suites — nothing selected, nothing averaged.** Each panel is one task on identical axes: the shaded square is the sampling region that task's own BDDL declares, the dots are the fifty initial states the benchmark ships, and the number beneath is the shipped spread. **Every LIBERO-Long task fills its declared box; no LIBERO-Object task does, and six are a single point.** The two rows are the whole of §2, and no tolerance, controller, policy or simulator enters the comparison. Generated by `paper/render_fig13_allsmall.py`.

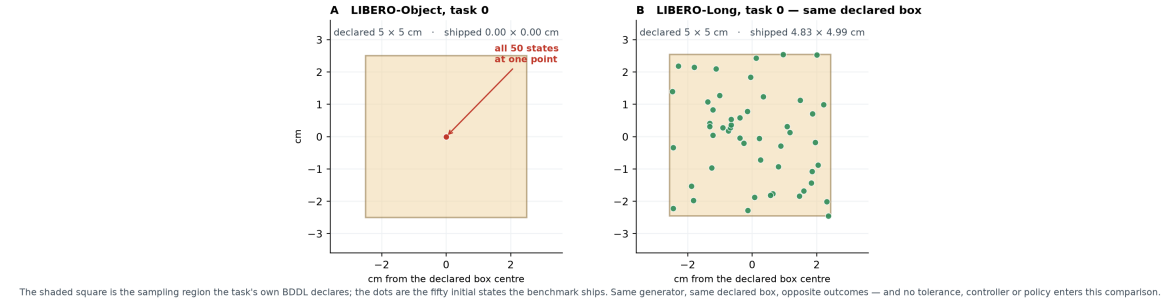


Figure 3: **The region a LIBERO task declares, against the states it ships.** The shaded square is the sampling box the task's own BDDL specifies; the dots are the fifty shipped initial states. **A** LIBERO-Object task 0: a 5×5 cm declaration, and all fifty states at one point. **B** LIBERO-Long task 0, same generator, the identical 5×5 cm declaration, drawn on the same scale: the states fill it. No tolerance, controller, policy or simulator enters this comparison — it is two shipped files. Generated by `paper/render_fig10_declared.py`.

2.1 The defect is exploitable, and we can now show it

A frozen placement is only interesting if it changes what a policy can do. Section 6 reports our failure to establish that, and this experiment settles the half of it that is settleable: run two policies on task 0 twice — once on the fifty states LIBERO ships, once on fifty drawn from the region its own task file declares — changing nothing else. Same weights, same seed, same trial indices, same machine, same package versions, one invocation of one script that runs the shipped condition, swaps the initial state files, and runs the repaired condition — back to back, not interleaved.

The obvious objection, and the arm that answers it. The repaired positions might simply be *harder* — in which case the drop measures difficulty, not exploitability, and the manipulation check proves nothing. Nothing in the first two rows of Table 4 separates those. So we ran the arm that does, and wrote both readings down before running it (Table 5). `fixed` keeps the controller, the states and the single static goal, and changes only where that goal comes from: one privileged look at reset instead of a constant compiled from the shipped files.

It scores 30/30**.** On the identical thirty repaired states where the lookup constant scores 10/30, a correct static goal never fails — 20 discordant trials, every one favouring `fixed`, $p = 2 \times 10^{-6}$. The repaired suite is not hard; a stale constant is simply wrong on it. That closes the confound, and it is why §2's defect is load-bearing rather than a curiosity about file contents.

A third arm we did not plan, and what it cost us. Table 5 came with an identity check: on a task whose target is pinned, we reasoned, `blind`'s compiled constant and `fixed`'s reset reading are the same float, so `fixed` on *shipped* states must reproduce `blind`'s 25/30 trial for trial. It did not — 10/10 against 6/10 on the same ten trials — and **the premise was what broke, not the harness**. Task 0 pins the target to a single point and *replaces the basket every episode*, over 2.9×2.8 cm. `blind` must supply both goals from shipped files, so it carries the *mean* basket; every other arm reads the true one. The two arms never received the same goal.

Three consequences, none of them cosmetic. **(i)** The lookup arm is *handicapped*, not favoured, and its 25/30 was scored while paying for a container it is not allowed

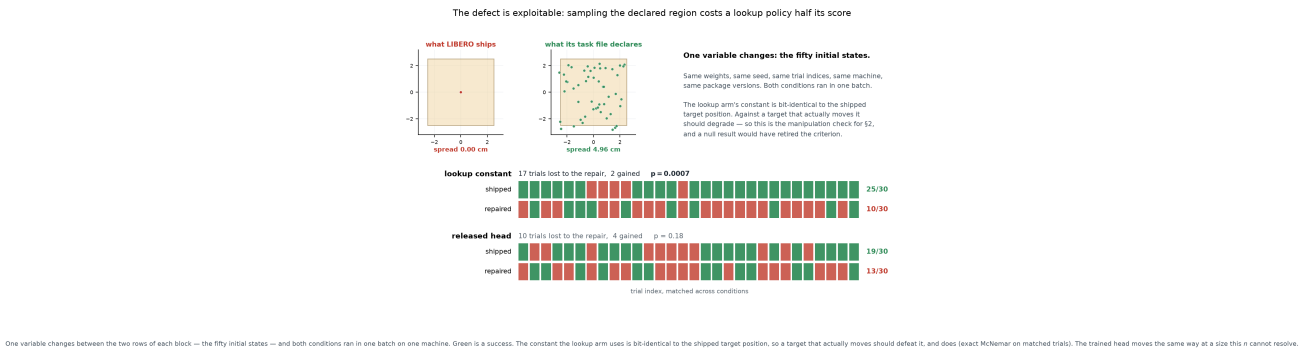


Figure 4: **The manipulation check, at the level of single trials.** Left: what changed — the fifty starting positions of the target, shipped and repaired, inside the 5×5 cm region the task file declares. Right: what happened, one column per trial index, green for success. The test is paired, so the raw outcomes are shown rather than two bar heights: a column green above and red below is a trial the repair cost. Generated by `paper/render_fig15_e1.py` from `results/e1_shipped_vs_repaired.json`.

goal supply	shipped	repaired	paired
lookup constant	**25/30**	**10/30**	17–2, $\mathbf{p = 0.0007}$
released head	19/30	13/30	10–4, $p = 0.18$
control: true target, read once at reset	10/10	**30/30**	vs lookup, repaired: 20–0, $\mathbf{p = 2 \times 10^{-6}}$

Table 4: **Sampling the region the benchmark declares costs a lookup policy half its score.** Task 0, n=30 per cell, exact McNemar over matched trial indices — one variable changes, the fifty initial states. The constant is bit-identical to the shipped target position, so against a target that actually moves it should degrade, and it does: 0.833 → 0.333, $p = 0.0007$. **This is the manipulation check for §2**: it establishes that the shipped states cannot defeat a table lookup and the repaired ones can, so the defect is load-bearing rather than cosmetic. **The control row closes the obvious hole**: handed the true target once at reset, the same controller succeeds on *every one* of the thirty repaired states, so those states are not harder — only the constant breaks on them. The trained head drops in the same direction, 0.633 → 0.433, which this n does not resolve ($p = 0.18$) — and which we would not read as exploitation even if it did, since the head was trained against the shipped states and the repaired ones are out of its training distribution. Drawn per trial in Figure 4; artifact `results/e1_shipped_vs_repaired.json`.

if `fixed` holds up on repaired	**if `fixed` collapses too**
the repaired states are not harder for a static goal, so the collapse of `blind` *is* the lookup failing	the repaired states are harder, whoever supplies the goal
⇒ the check in Table 4 stands	⇒ E1's drop is confounded, and we say so
outcome: 30/30 — **the left column**	

Table 5: **Written before the arm was run**, so the reading could not be chosen afterwards — which is error 7 of §11. Both branches were implemented and calibrated on synthetic cells first (`scripts/analyze_e5.py`); the script also refuses a verdict on an incomplete cell, tested on a short cell whose partial data would have rejected.

to look at: across those ten trials its failures carry a mean basket displacement of 1.59 cm against 1.01 cm for its successes (exact permutation $p = 0.0095$). **(ii)** The comparison that isolates E1's confound is therefore `fixed` shipped against `fixed` repaired, where the basket distribution is bit-identical — the repair rewrote state columns 10–11 and the basket lives in 17–18 — and that is the test reported above. **(iii)** It is an error of ours, and we record it — but *outside* the taxonomy of §11, on purpose. Those nine share one structure: a correct instrument, a chosen slice. This one is a false premise about the apparatus, a different animal, and folding it in to make the count larger would blunt the only claim that taxonomy makes. What it is instead is the first of our errors that no human found: a check written to catch a harness bug fired on our own premise, and cost us a sentence already typed into this section. Automation caught none of the other nine; it caught this one because the prediction was *deterministic* — same float, same states, same controller — so a single disagreeing trial falsified it outright, with no statistics in the way. That is the shape of claim a machine can check, and it is worth knowing which of one's claims have it.

One reading we are not entitled to. Inside Table 4 the lookup arm *leads* the trained head on shipped states (25/30 against 19/30) and *trails* it on repaired ones (10/30 against 13/30). That interaction is exactly what this paper argues, and it does not survive its own test: the difference of differences is +0.30 at a permutation $p = 0.14$ (200,000 draws), and neither within-condition contrast separates ($p = 0.15$ shipped, $p = 0.63$ repaired). We report the direction and claim nothing from it. Writing it down was error 4 the first time; leaving it out would be error 9.

Two honesties about this cell. The released head reproduces here at 19/30 = 0.633 against its historical 0.700, inside that interval, so the build is sound. But

the lookup arm scored 25/30 = 0.833 where an earlier box measured 6/10 = 0.600 — same code, different torch. We report both and adopt neither, because §9 is about exactly this and it would be absurd to quietly keep the flattering one. Nothing above depends on the cross-build comparison: both conditions of Table 4 ran back to back in one invocation on one machine.

That leaves the question this paper exists to answer. A frozen placement is only a problem if a policy can exploit it, and whether it can is a question about the *embodiment* as much as about the files. §3 makes that precise; §6 reports our attempt to demonstrate it, which failed.

3 When a frozen placement is exploitable

§2 establishes that one suite ships a constant where its task files ask for a distribution. This section asks when that is *fatal* — when the frozen placement is close enough, relative to what the robot can resolve, that a table indexed on the task alone reproduces whatever a perceiving policy would have supplied.

3.1 Definitions

Fix a task t that ships a finite set S_t of initial states. A task requires the policy to localise one or more objects; write O_t for that set, which the benchmark declares (LIBERO ships it as `obj_of_interest`), and $x_t^o(s) \in \mathbb{R}^2$ for object o's table position in state s. All norms are Euclidean unless subscripted. §B recomputes everything in ℓ_∞ — the norm A1 is naturally stated in — where the count elsewhere rises to 3/30, the third task is a movable target rather than a fixture, and the separating window no longer contains the δ we use. The choice of norm is one this result is sensitive to.

Definition 1 (Placement radius)**.** *The* placement radius *of object o in task t is the radius of the smallest disc containing every shipped position,*

$$R_t(o) \;=\; \min_{c \in \mathbb{R}^2} \; \max_{s \in S_t} \|x_t^o(s) - c\|.$$

R is the quantity the next proposition needs, and it is the quantity because a single constant serves o *if and only if* $R_t(o) \le \delta$ — the minimising c *is* that constant.

The distinction from the *diameter* $D_t(o) = \max_{s,s'} \|x_t^o(s) - x_t^o(s')\|$ is not cosmetic. $D \le \delta$ implies $R \le \delta$ but not conversely: $R \ge D/2$ always, and Jung's theorem gives $R \le D/\sqrt{3}$ in the plane, so $D > \delta$ licenses nothing. An earlier version of this paper used D to claim that no task outside LIBERO-Object was admissible; recomputed on R at that version's $\delta = 2.5\,\text{cm}$ the count is 27 of 38, not 10.

Definition 2 (Lookup-admissibility)**.** *Object o of task t is* lookup-admissible at δ *if $R_t(o) \le \delta$. A suite is* lookup-admissible *at δ if every object of every task is.*

Which objects, and why it matters. Every LIBERO task requires at least two: the thing to be moved and the place to put it. Call *o identity-bearing* if it is what individuates the task from its siblings — the grocery in "pick up the *alphabet soup* and place it in the basket" — and *shared* otherwise. LIBERO-Object's ten tasks differ only in the identity-bearing object; all ten share one basket.

We report the two separately, because the split is the finding rather than a technicality.

Measured over both, *no* LIBERO-Object task is admissible: the shared basket has $R \in [1.67, 1.98]$ cm and moves every episode. An earlier version reported the identity-bearing object alone and called the suite admissible.

How we pick the identity-bearing object, and how much it matters. We take it to be the first entry of the task's declared `obj_of_interest`. That is *not* the same as "the object unique to this task": in 22 of 40 tasks the first entry is shared with a sibling task, and LIBERO-Object is the one suite where the two rules coincide.

Under the stated semantics instead — take the entry unique to the task — the count elsewhere goes from 2/30 to **8/30**; taking the last entry (the destination) reverses the result entirely, to 0/10 against 16/30. The verdict is a function of a labelling choice, and we report all three (`results/admissibility.json`).

Part of the separation is built into how LIBERO is organised. LIBERO-Object's ten tasks are individuated by *which object*, so the suite has no need to move it. LIBERO-Spatial's are individuated by *where the bowl is*, so its identity-bearing quantity *must* vary or the suite would not exist. To that extent Table 6's contrast restates the four suites' design intents rather than discovering a defect in them.

What is not built in is the magnitude. Nothing about "individuate by object" requires the position to be frozen to one `float64` value when the task file declares a 5×5 cm region to sample from; LIBERO-Object could randomise placement freely and still individuate by identity, and LIBERO-Spatial demonstrates that a suite can carry both. **The finding is the gap between the declared region and the shipped states, not the ordering of the four suites.** The ordering is what the metric computes; on its own it would have been unsurprising.

What we do not claim. That a low radius *causes* a policy to memorise; that any published policy exploits it; or that admissibility on the identity-bearing object is sufficient for a constant to solve a task. §6 shows the last of those failing on our own stack, decisively.

Assumption A1 (δ-robustness). Supplying the controller a constant c with $\|x_t^o(s) - c\| \le \delta$ leaves the episode

outcome unchanged, for every s. A1 is an empirical property of an embodiment, not a definition. §6 measures it directly and finds a radius of about 1.4 cm on this controller — and finds that the radius is not the same number on every task, which is a real limitation of what follows.

Figure 5 draws both the definition and the proposition that follows.

Proposition 1 (Lookup sufficiency)**.** *If $R_t(o) \le \delta$ for every object of every task in a suite and A1 holds at δ, then a policy that reads only the task index, emits the corresponding constants, and is otherwise driven by a controller whose behaviour depends on the supplied goal and not on object identity, achieves on that suite whatever that controller achieves given the true positions.*

Proof. Take c_t^o to be the centre of the minimum enclosing disc. By Definition 1, $\|x_t^o(s) - c_t^o\| \le R_t(o) \le \delta$ for every s, so $t \mapsto (c_t^o)_{o \in O_t}$ is a lookup table meeting Definition 2. A1 applies at each state, giving the same outcome as the true positions; averaging over S_t gives the claim. □

The mathematics is trivial and we do not pretend otherwise. All of the content sits in A1, which is why §6 is about measuring A1 rather than about the proposition.

Corollary 1 (No score can separate them)**.** *On a suite satisfying the hypotheses, no function of episode outcomes distinguishes the lookup policy from any policy that supplies the controller a position within δ of the truth on every shipped state.*

Corollary 1 is a statement about a *comparison class*, and the class is narrow: it contains only policies that are accurate everywhere. No policy in this paper is in it, ours least of all. So the corollary bounds what a score *could* certify; it does not license reading any particular pair of measured cells as confirming it. A measured pair of cells can neither confirm nor refute it; treating them as though they could is a mistake we have made in both directions.

3.2 Measurement

Extracting a position from a shipped state is not mechanical — a naive fixed-column layout is wrong for every suite containing an articulated fixture — so we compile each task's model and read `jnt_qposadr`; Appendix A gives the procedure and the cross-check.

Two LIBERO-Goal tasks name a fixture with no free joint — a cabinet drawer, a stove. A fixture that never moves has $R = 0$ and is the *most* lookup-admissible object there is, so they are counted below and identified; an earlier version excluded them as having "no placement to measure".

Table 6 is the measurement, and it says one thing precisely.

> **On the object that individuates its tasks, LIBERO-Object is lookup-admissible on all ten. Among the other thirty tasks, two are — and both are fixtures that never move at all.**

	identity-bearing R (cm)		shared R	admissible
suite	mean	range	range	at 1.4 cm
Object	**0.30**	0.00–**1.17**	1.67–1.98	**10/10**
Spatial	2.15	1.49–4.49	1.66–2.01	0/10
Goal	1.48	0.00–2.48	0.00–1.70	2/10†
Long	3.10	2.85–3.28	0.00–3.17	0/10

Table 6: Placement radius of every LIBERO task, computed from the shipped initial states alone — no policy, no simulator, no training. The last column counts tasks whose identity-bearing object a single constant serves at the tolerance we measure for this controller (§6). **LIBERO-Object pins all ten of the objects its tasks are individuated by; elsewhere two of thirty**, and †both of those are the fixture tasks — a drawer and a stove — which have $R = 0$ because they are bolted down, not because a randomisable placement was frozen. On the shared container the ordering **reverses**: LIBERO-Object pins it in 0 of 10 tasks and LIBERO-Goal in 6 of 8, whose shared objects are fixtures. So LIBERO-Object is not the suite that pins the most — it is the suite that pins the object its tasks are *named* by, which is the distinction this paper is about and the reason the two columns are reported separately. Artifact: `results/admissibility.json`.

The threshold is not doing the work. LIBERO-Object's largest identity radius is 1.171 cm; the smallest among the other suites' twenty-eight *movable* targets is 1.492 cm. Every δ in that gap returns the same verdict, and the radius we measure for this controller (≈ 1.41 cm, §6) falls inside it. This is a coincidence, and we treat it as one: the window is 0.32 cm wide, and at a 2 cm tolerance most of LIBERO-Spatial and LIBERO-Goal would be admissible as well. Appendix C gives the whole curve. **The verdict is conditional on a measurement we make from a single n=10 cell**, and a reader should weight it accordingly.

How pinned is pinned. In 6 of Object's 10 tasks the target's start position takes exactly *two* `float64` values, one unit in the last place apart on a single axis — constant to the precision the format can express. The other four have $R \in [0.60, 1.17]$ cm, consistent with the subcentimetre jitter in Figure 6A.

Across tasks the structure is tighter still: the ten targets occupy two table positions 22.1 cm apart, five tasks each (Figure 6A), so a two-constant table covers the identity-bearing half of the whole suite.

4 A glass-box stack

The artifact is an instrument, not a contribution (Table 7). It is built to be traceable end to end, and its size (30.2 M deployed) is what makes an end-to-end trace feasible rather than a claim about efficiency.

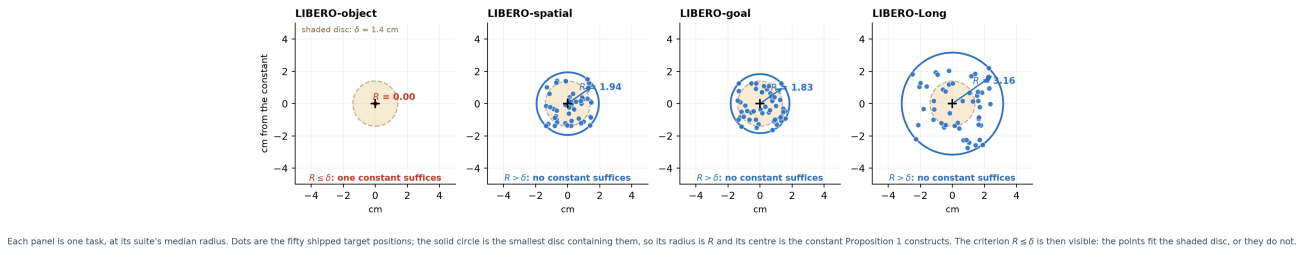


Figure 5: **Definition 1 and Proposition 1, drawn.** One task per suite, each at its suite's median radius. Dots are the fifty shipped target positions; the solid circle is the smallest disc containing them, so its radius *is* R and its centre *is* the constant the proof of Proposition 1 constructs. The shaded disc is the tolerance δ. The criterion $R \leq \delta$ is then a picture: the shipped points fit inside the shaded disc, or they do not. Generated by `paper/render_fig11_radius.py`.

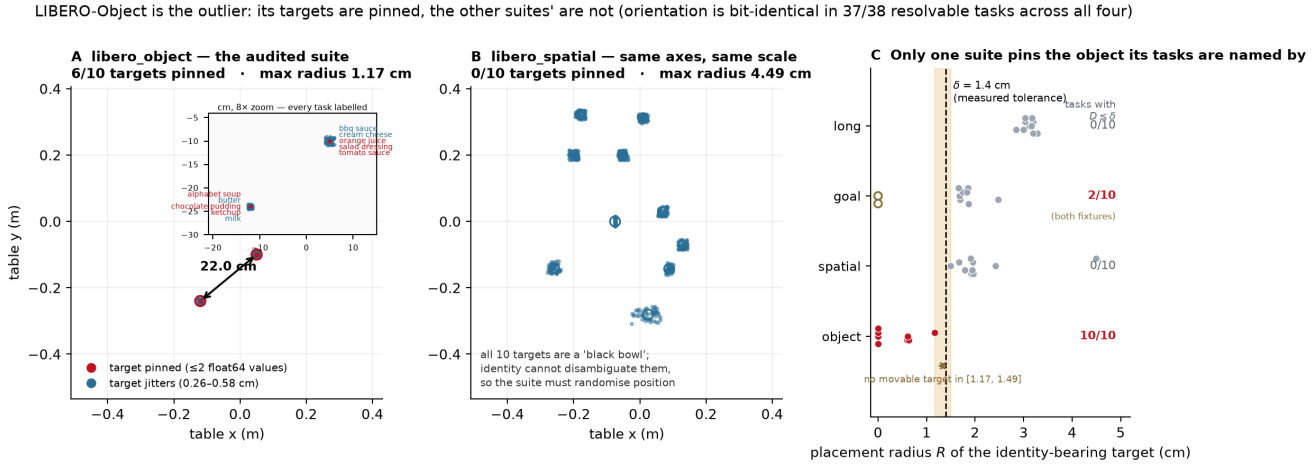


Figure 6: Placement forensics. **A** LIBERO-Object's identity-bearing targets, 10 tasks × 50 shipped states; six collapse to a point and the ten task means occupy two clusters 22.1 cm apart. **B** LIBERO-Spatial on identical axes: every target scatters, and it must, because all ten of its targets are a "black bowl" and identity cannot disambiguate them. **C** placement radius per task against the tolerance measured for this controller. Generated by `paper/render_fig1_placement_forensics.py`; SHA-256 digests of every init-state array are in `results/suite_forensics_columns.json`.

component	params	trained	deployed
YOLO-World-S detector	13.0 M	never	frozen
world model (TRM)	9.97 M	stage A	frozen, inert†
trunk heads	7.0 M	stage A	frozen
goal heads + gates	0.24 M	stage B	frozen
deployed	**30.2 M**	17.2 M trained	all frozen

Table 7: Parameter ledger. There is *no* language model: the open-vocabulary detector's own text tower supplies every text embedding, so one frozen network is the only vision encoder *and* the only text encoder. †Inert in the one cell we have ablated: a persistence baseline reproduces the held-out cell exactly. We state it for that cell only.

Text is parsed into source/target phrases; the detector is prompted with them and returns a source box. A grasp head maps (uv, conf, proprio, box_emb, frame_emb) to a world grasp point — note the absence of any text argument, which is Figure 10's point and §9's architectural evidence. A place head maps the command embedding to a place (x, y), latched once per episode inside `reset()` and never re-read. A non-learned state machine converts goals into servo commands within a ±6 cm probe envelope.

Three words, used throughout.

- **cell** — one evaluation: a fixed (head, flags, seed, n) run to completion, always quoted with its n and its interval.
- **band** — the set of shipped initial states a cell replays. The harness fixes it; it is not sampled.
- **burned** — a band we have looked at and let influence a choice. From that moment it no longer measures generalisation, and no amount of re-running un-burns it.

Protocol. LIBERO at commit `8f1084e3`, MuJoCo 2.3.7, robosuite 1.4.1, torch 2.8.0, ultralytics 8.4.115, numpy 2.2.6, Python 3.10, `MUJOCO_GL=osmesa` — §9 explains why that last entry is not boilerplate.

Which states compose a cell is derivable from its command line, not logged. The harness sets

`trial_seed` = `seed` · 1,000,003 + `trial` and indexes state `trial_seed` mod 50; since 1,000,003 ≡ 3 (mod 50), trial i at seed s replays state $(3s + i)$ mod 50:

cell	seed, n	states
dev	0, 10	0–9
held-out	20, 10	10–19
held-out n=50	20, 50	**0–49**
untouched	40, 30	**20–49**

Table 8: Which of the 50 shipped initial states composes each evaluation cell, derived from the harness's seeding rule rather than logged. Because `trial_seed` = `seed` · 1,000,003 + `trial` and 1,000,003 ≡ 3 (mod 50), trial i at seed s replays state $(3s + i)$ mod 50. The bolded rows are the ones that matter: an n=50 cell is the entire state set and therefore *contains* the dev and held-out bands rather than being disjoint from them.

The answer is not flattering: **the n=50 cells are not disjoint from the dev and held-out bands — they contain them.** Fifty trials over fifty shipped states is the whole set. The untouched band exists to supply one clean cell, and §8 reports it.

5 A behavioural test, and why it cannot decide

§3 says a suite with $R \leq \delta$ admits a lookup solution. Whether one actually *works* is a separate, empirical question, and this section reports our attempt to answer it.

The outcome first. **The experiment does not decide the question, and we can say why: the instrument tops out at** 16 **successes in** 100 **trials, eight of ten tasks score zero for every arm we ran, and the two that do score are separated by the object rather than by the goal.** What follows is the design, the result, and the diagnosis; §6.1 names the experiment that would decide it.

This is not in tension with the 25/30 of §2.1. That cell is one task at n=30, chosen because it is one of the two on which anything scores at all; the ceiling quoted here is over ten tasks, eight of which score zero for every arm we ran. The suite-level number is low *because* of those eight, and it is the reason a suite-level comparison cannot decide anything.

The vehicle is our own stack (§4): a learned goal head that emits a world grasp point, driving a non-learned pick-and-place controller. That factorisation is what makes the experiment clean — we replace the *goal supply* and change nothing else. Figure 7B lists the five arms and what each may read. Two need a word. **random** draws from the observed extent of the scene's objects, re-drawn per episode — the controller's floor. **reset-oracle** reads the true position *once* at reset and holds it; on a task with $R = 0$ that is numerically a per-task constant *for the target*, but it still reads the simulator, so it is **not** Proposition 1's policy — and the equivalence stops at the target, because it also reads the true *container*, which LIBERO-Object re-places every episode and which **blind** must supply from a shipped mean (§2.1).

The floor is zero. With an uninformed goal the machinery scores 0/10 [0.00, 0.28]: eight discordant pairs, all favouring the learned head, exact McNemar $p = 0.0078$. **The controller cannot do this task without a goal.** That is the control that makes the rest of the section mean anything.

The strictly-blind policy. The **blind** arm takes the task index, opens the shipped initial-state files, and emits two constants — the mean shipped position of the identity-bearing object and of the container. On task 0 those are $(-0.1200, -0.2400, 0.0150)$ and $(0.0023, 0.2605)$.

On this task the grasp constant is not an approximation of the target position; it *is* the target position. All fifty shipped states place the alphabet soup at a single (x, y) — the fifty rows reduce to one — so whatever the blind arm fails at, it cannot be error in where it thinks the object is.

It scores 6/10 [0.31, 0.83], against the released head's 8/10. We report no inference from that pair. At n=10 an exact paired test with two discordant pairs returns $p = 0.50$, which is the largest value the test can return with any discordance at all; a power calculation gives 0.058 against the observed effect. **The cell resolves nothing.** It does not establish indistinguishability, and the corollary does not rescue it: Corollary 1 quantifies over policies accurate everywhere, and an 8/100 policy is not one.

Three caveats on the arm itself, none of which we found ourselves. The constant is the *mean* over all fifty shipped states, which contains the ten this cell replays — it is fitted, very weakly, on its own test set. The arm reads the object's identity from the live scene's body registry rather than from the task index, so "reads only the task index" overstates it. And it requires a live simulator to start at all.

The *control* claim survives all three — with an anchor set, the goal machine is driven by proprioception and the constant, and the grasp head is never called — but the *blindness* claim as we first wrote it did not.

6 The diagnosis

Every cell above is one task. Run on all ten (Figure 8), the blind constant scores 16/100 and the released head 8/100.

Every cell uses seed 20 and so replays states 10–19, the held-out band of Table 8. That band is burned for task 0 by the selection loop of §7; for tasks 1–9 no round of selection ever reached it.

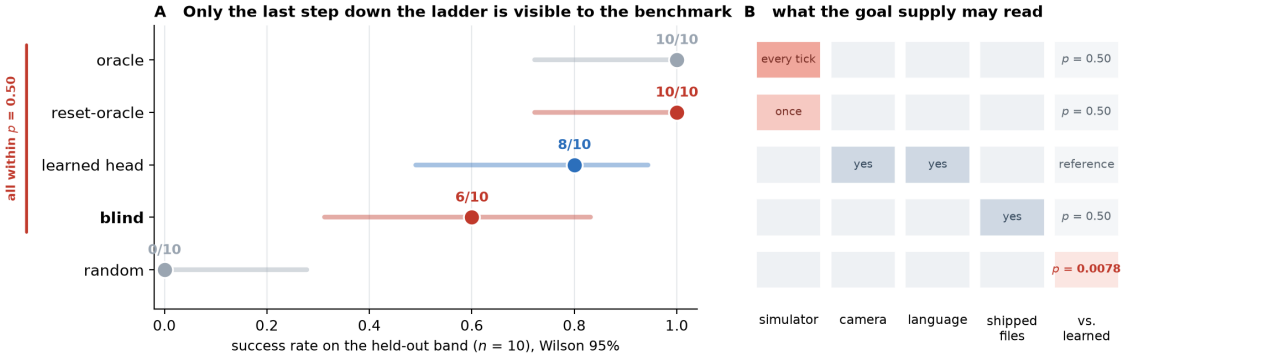


Figure 7: **The goal-supply ladder**, on task 0. Five arms, one controller, one trunk, one protocol, one set of initial states; the only difference is where the goal comes from. **B** is the experiment: descending the ladder removes information. **A** is the result — and the honest reading of it is that at n=10 this cell resolves almost nothing. The bracket marks arms the cell fails to separate, which is a statement about its power, not a finding of equivalence. Artifacts: `results/pod_cells.json`, `results/blind_cells.json`.

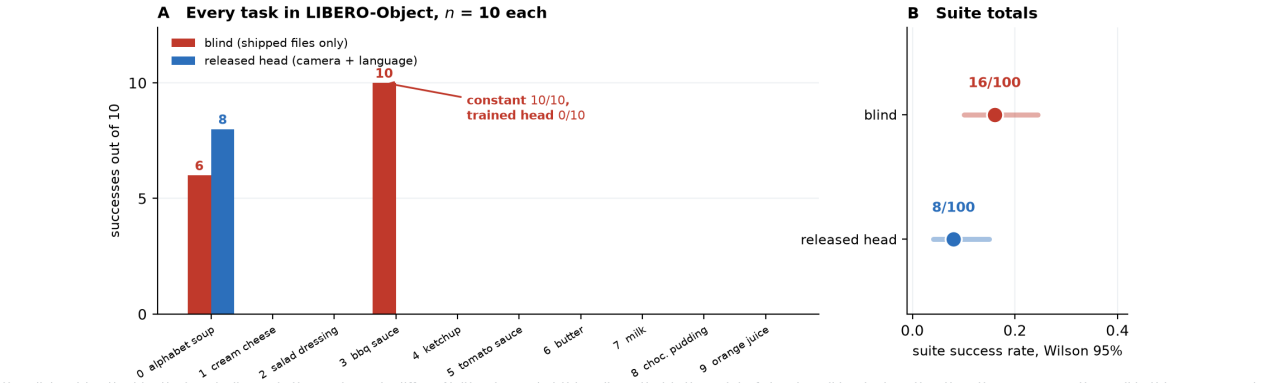


Figure 8: **All ten LIBERO-Object tasks**, n=10 each, every trial recorded; both policies drive the identical controller. Eight of ten tasks are 0–0. The suite totals in **B** are shown with i.i.d. Wilson intervals, which are *too narrow* here by a design effect of 8.4 (§6); they are drawn as reported rather than as believed. Artifacts: `results/suite_cells.json`, `results/suite_inference.json`.

An earlier draft led with the 16-against-8 pair, paired the 100 trials, ran an exact McNemar, obtained $p = 0.039$, and concluded that a task-indexed constant significantly beats a trained vision-language policy.

That inference is wrong, and the error is instructive.

The unit of analysis. All twelve discordant pairs live in two of ten tasks, and ten of them in one (Table 9). Within that task the ten "trials" are near-identical replays: blind terminates at steps 210–214 (sd 1.08), the head runs to the 300 cap on all ten. They are one deterministic contrast counted ten times.

At the task level — the unit the design actually randomises over — blind wins one task, loses one, and ties eight: exact sign test $p = 1.0$, exact sign-flip permutation $p = 1.0$, cluster bootstrap on the difference $[-0.06, +0.30]$. The design effect is 8.4, so the effective n is about 12, not 100.

The constant did not produce the successes. This is the sharper finding, and it comes from the same run. An earlier version compared the ten constants *in the table plane* and said they were identical to within a millimetre. The arm is handed a 3-vector, and in three dimensions the two position groups differ by 40.0 and 35.0 mm on z. That sentence was true in 2D and false as written.

What survives is one comparison (Table 10), and it needs no statistics. **Tasks 2, 5 and 9 are handed a constant 0.469 mm from task 3's in full three dimensions. They score 0/10. Task 3 scores 10/10.** Half a millimetre of goal, against a tolerance of centimetres, cannot separate a total failure from a clean sweep.

What varies across those tasks is the object, not the number. So the blind arm's two non-zero cells are evidence that *this controller can grasp two of ten groceries*, and not evidence about lookup. **We withdraw the suite-level claim.**

unit	test	result	
trial (n=100)	exact McNemar	10–2, $p = 0.039$	*invalid*
task (n=10)	exact sign	1–1, 8 ties, $p = 1.0$	valid
task (n=10)	permutation	$p = 1.0$	valid
task (n=10)	cluster bootstrap	$[-0.06, +0.30]$	valid

Table 9: The same comparison at two units of analysis. Trials within a task share an object, a scene, a goal supply and a near-deterministic trajectory, so the trial-level test is anti-conservative. **The task-level tests do not show equivalence — they show this design cannot resolve the question in either direction.** With eight ties the sign test has two informative units and its smallest attainable p is 0.5; the permutation test has four arrangements and the same floor. Reading $p = 1.0$ as "no difference" would repeat, in the other direction, the error this section withdraws. The design effect (≈ 8, bootstrap $[5.4, 10.0]$, estimated from one non-degenerate cluster) is reported as a description of the clustering, not as a correction to any p. Artifact: `results/suite_inference.json`.

task	distance from task 3's constant (3D)	blind
3 `bbq_sauce`	—	**10/10**
2 `salad_dressing`	0.469 mm	0/10
5 `tomato_sauce`	0.469 mm	0/10
9 `orange_juice`	0.469 mm	0/10

Table 10: **The goal supply does not explain the outcome.** Four tasks receive the same constant to within half a millimetre in all three axes and score 0, 0, 0 and 10 out of ten. This is a deterministic contrast, not a statistical one. We previously also offered a correlation between placement radius and success ($r = +0.52$) as evidence here; **we withdraw it**. Its exact permutation p is 0.13 and deleting task 3 flips it to -0.25 — the sign is one task, the error this section reports. Artifact: `results/constant_attribution.json`.

Where A1 does break, we can see it. One place in this data does behave the way the proposition expects. On task 0 the blind arm reaches the object to within 2 mm on all ten trials — the grasp constant is exact there — and the four failures are the four trials whose *basket* sits furthest from the constant, occupying ranks 6, 8, 9 and 10 of ten (exact Mann-Whitney $p = 0.019$, Figure 9). Task 3 tolerates 1.91 cm and never fails. That gives the ≈ 1.4 cm radius §3 uses, and shows it is task-dependent.

Two caveats we owe a reader, because this is the number §3 leans on. The hypothesis was formed *after* a pre-registered one failed, on the same ten points. And the covariate is confounded with trial order: the outcome is exactly $\mathbf{1}\{\text{trial} \leq 5\}$, trial i replays state $10 + i$, and displacement correlates with state index at $\rho = 0.65$, so trial index alone predicts better ($p = 0.0095$). At n=10 the design cannot separate them.

6.1 What would settle it

Displace the constant by r and sweep r: if the score is flat in r, the arm never used the number. We implemented it (`--goal-anchor-jitter-cm`) and the compute window closed before it ran. Until it does, the strongest statement the suite supports is the negative one above.

What survives. Two things, First, on task 3 a constant scores 10/10 where the trained head scores 0/10 — a single task-level event, so we quote no p for it, but one worth stating: the head is not merely worse there, it never succeeds.

Second, and independent of any of this, **eight successes in a hundred trials, every one on the single object the head was trained for**. Our artifact is a one-object solution that a suite-level score would have reported as a policy. The number a reader should carry away is not 0.700; it is 0.700 on task 0, with 0.000 next to it nine times.

And it bounds the audit in advance. Whatever §7–§8 bought, it was not a general policy. We put this section before the audit so that no reader reaches the repairs holding a larger idea of the artifact than the artifact supports.

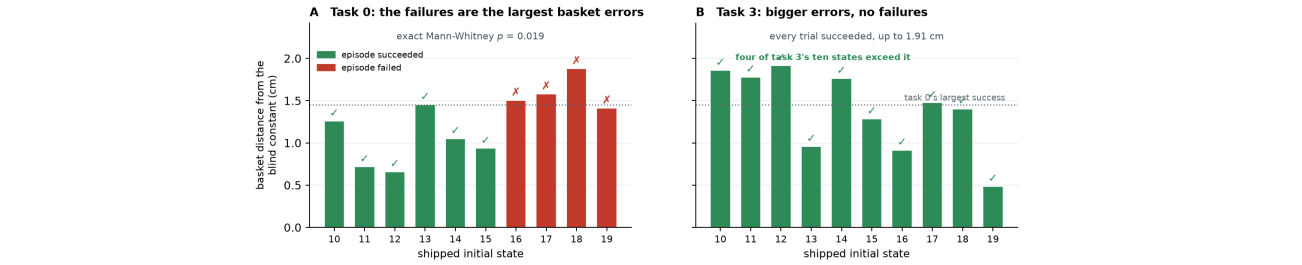
7 The audit: four layers

"The policy memorised the location" is not one hypothesis but four, at four stages: causal confusion [1] spread across a pipeline rather than confined to a network. Two of the four are not in the model at all, and those two are the ones a black-box perturbation study cannot reach.

Table 11 states all four with their repairs. Three are described here; the fourth, being the one the instruments found rather than confirmed, gets its own subsection.

Layer 1: the expert's calibration constant. The scripted demonstrator carried a hand-eye offset fitted once, on a scene whose target never moved. That constant therefore *encoded* the pinned placement, and every demonstration inherited it — so the corpus taught the pose before any network saw a pixel. Detected by reading the expert, not the policy. Repaired by re-baking ten episodes with the source object displaced, which breaks the constant's validity and forces the demonstrator to look; the repaired corpus lifts the dev cell from 0/10 to 7/10 on the same episodes.

Layer 2: the selection loop. Our own procedure, not the artifact's: at each round we chose a checkpoint using cells that later rounds reused, so selection pressure accumulated on one band of initial states. This is invisible to any measurement of the model and is the reason §8 runs an untouched band at all. It is *bounded* rather than repaired — one cannot un-look at data — and the bound is that thirty never-inspected states score within noise of the burned ones.



The blind arm is given one basket position per task, read from the shipped files. Within task 0 the error in that constant predicts which trials fail; across tasks the tolerance is not the same number — which is assumption A1 being measured rather than assumed.

Figure 9: **The one place the constant demonstrably matters. A** On task 0, the trials that fail are those whose basket sits furthest from the single constant the blind arm was handed. **B** On task 3 the same quantity runs higher and every trial succeeds, so the tolerance is not one number across tasks. This is the measurement §3's $\delta \approx 1.4\,\text{cm}$ comes from; see the confound caveats in the text. Artifact: `results/blind_failure_attribution.json`.

layer	where	evidence	repair	verification / status
1. expert calibration constant	scripted expert (non-model)	hand-eye constant encodes the pinned placement	placement teleportation	0/10 → 7/10 dev, paired. **Repaired**
2. selection loop	our own procedure (non-model)	checkpoints chosen on cells later rounds reused	process fix; bands frozen	untouched band 20/30 vs 0.700: no detected inflation. **Bounded, not removed**
3. grasp-head proprio.	`GraspPointHead`	prediction tracks the eef, flat under uv sweeps	same teleported episodes	attribution flips to visual. **Repaired**
4. place-head cmd→location	`PlaceHead`	correct basket for the trained command, 14.5/13.0 cm off for the other two; swap 8/10 → **0/20**, p=0.0078	command coverage	place *point* 14.15 → 0.78 cm **repaired**; swap *behaviour* 0.767 → 0.433, p=0.031 — **halved, not fixed**
(separate) appearance drift	deployment viewpoints	NN-cosine gap on the policy's own trajectory	DAgger on own viewpoints	3/50 → 22/50 vs 3/50 control, $p < 10^{-4}$. **Repaired**

Table 11: Master audit table. **Provenance, since this is the paper's largest table without a per-row artifact:** the swap cells (8/10 → 0/20, 23/30 → 13/30) and the untouched band (20/30) have shipped per-trial records in `results/pod_cells.json`; the attribution profiles are in `results/attribution_profiles.json`. The repair figures — layer 1's 0/10 → 7/10, layer 4's place points, and the DAgger row's 3/50 → 22/50 — were measured in earlier sessions and their per-trial records are *not* in this repository. They are reported here as cited history, not as cells a reader can recompute. Layers 1–4 are the placement-memorisation layers; the last row is a drift case study, *not* one of the four; it appears here only because its repair is easily mistaken for one of theirs. Two rows are deliberately not marked repaired.

Layer 3: the grasp head's proprioceptive shortcut. Substitution attribution showed the predicted grasp point tracking the end-effector and barely moving under *uv* substitution: the head had learned to predict where the object is from where the arm is, which on a pinned suite is a sufficient statistic. The same teleported episodes repair it, and the released head's profile inverts — it now moves further on its visual channels (3.43/6.57 cm) than on proprioception (1.93 cm). McNemar $p = 0.0078$.

Telemetry places the failure *downstream of approach*: the policy still reaches the true object as closely as baseline while the grip-close rate falls from ∼0.67 to ∼0.26. Driving the detection prompts and the place-head embedding from *different* instructions isolates the collapse to one channel, with no interaction.

7.1 Layer 4: where the language goes

Swapping the instruction while holding the scene and the success predicate fixed collapses the released head from 8/10 to **0/20** $[0.00, 0.16]$; on the ten trials that pair with the reference cell, eight discordant pairs all one way, exact

Combined with the architecture: **object selection, approach and grasping run with no language input at all, and the entire language channel is a single cached** (x, y). Ask this policy for the butter and it finds, approaches and grasps the alphabet soup at baseline rate; it fails only when that one coordinate is wrong.

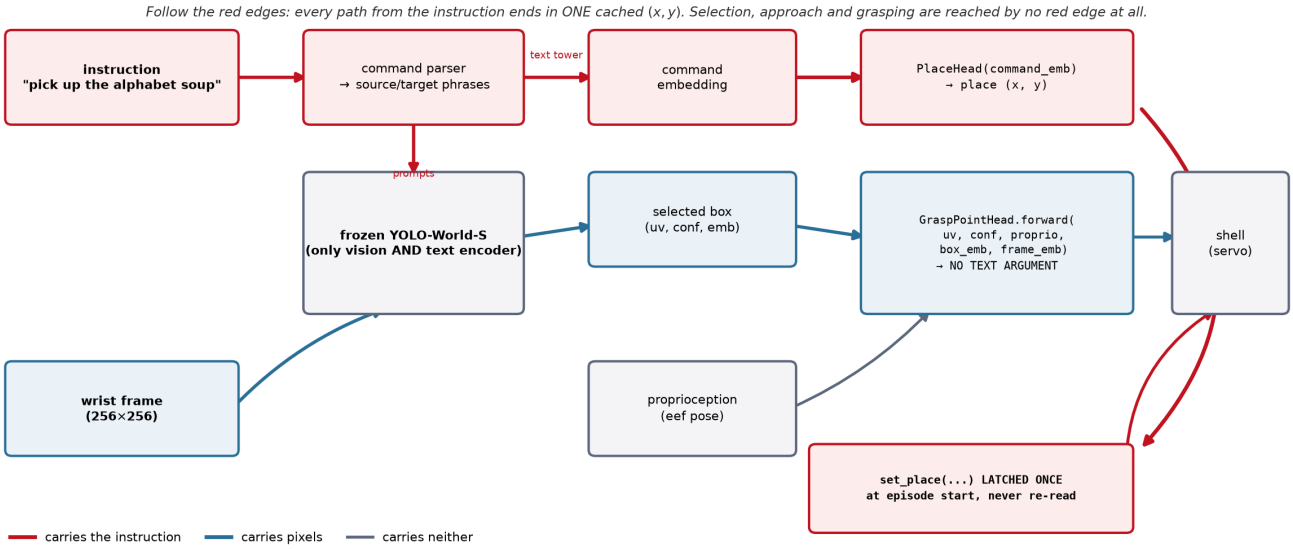


Figure 10: Every edge coloured by what it carries. The architectural half of the argument is a code fact: `GraspPointHead.forward` takes no text argument, so no red edge reaches object selection, approach or grasping, and every red edge terminates in one (x, y) latched once per episode and never re-read.

8 Repairs, and one we withdraw

Three of the four layers admit a repair, and we report what each bought. One claim from the previous version does not survive being run at power, and we retract it here rather than leave it standing.

The untouched band. Both n=50 cells contain the burned states, so we ran the released head on states 20–49, which no selection look ever reached, with the prediction recorded before the run ([0.50, 0.75], falsifiers named in both directions). Result: $\mathbf{20/30 = 0.667}$ [0.49, 0.81], against the published 0.700 (35/50) a difference of −0.033, Newcombe [−0.24, +0.16]. That interval assumes independent samples and the samples are *not* independent: the n=50 cell spans states 0–49 and so contains all thirty untouched states.

The interval is therefore conservative in the wrong direction. And the clean comparison does not need a new run at all — it is subtraction, which we had missed: the n=50 cell contains the thirty untouched states, so the twenty burned ones scored $35 - 20 = \mathbf{15/20 = 0.750}$.

Against them the untouched band gives 0.667 versus 0.750, a difference of $\mathbf{-0.083}$ — two and a half times the −0.033 we reported against the superset — with Fisher exact $p = 0.75$. The burn is still not detectable at this n, but the honest effect size is the larger one, and the direction is the one that flatters us less.

We claim exactly this much: not that the burn is zero, but that it is small enough to hide inside ±0.20, which is what n=30 buys.

A repair we withdraw. Command coverage was reported as repairing the instruction-swap behaviour on the strength of 3/10 against a 4/10 baseline at $p = 1.0$. That is an equivalence claim from an underpowered cell, on a band where the head was already mostly failing — and a swap cannot visibly break a cell that is already broken. Re-run on a band where the coverage head scores well, and taken to n=30 (Table 12):

head	baseline	swapped	paired McNemar
released head	8/10	**0/20**	8–0, $p = 0.0078$
coverage	23/30	**13/30**	12–2, $p = 0.031$

Table 12: Substituting a different task's instruction, on a band where each head scores well and at a power the original cell did not have. Both heads lose accuracy, so command coverage does not eliminate the instruction dependence it was built to remove; it halves it. Paired McNemar over matched trial indices (same seed, same initial states, same renderer — only the instruction differs), exact two-sided.

At n=10 this contrast was $p = 0.125$ and we would have called it unresolved; at n=30 it is $p = 0.031$. **The coverage head collapses too.** A swap cannot visibly break a cell that is already mostly broken, which is why the original 4/10 baseline hid the effect.

What replaces the withdrawn claim is more useful than it was. Coverage does not eliminate the failure, it roughly *halves* it: the released head loses its entire cell (0.800 → 0.000, every discordant pair one way), the coverage head about a third (0.767 → 0.433). And the repair to the place *point* is not in doubt — 14.15 → 0.78 cm (Figure 11), measured directly rather than inferred from behaviour.

So: **command coverage fixes where the head aims and does not fully fix what it does.** A residual command→location dependence survives training on all

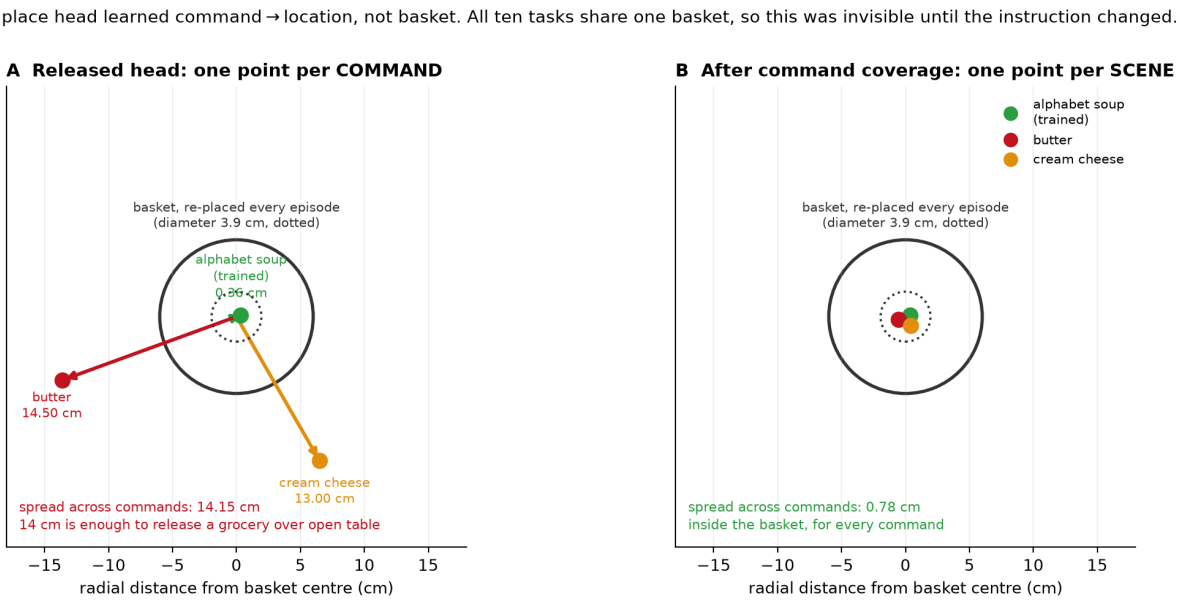


Figure 11: Layer 4 drawn rather than argued. The released head asked for three objects emits three place points, 14.5 and 13.0 cm from the basket. The basket is *not* fixed — it is re-placed every episode, with a placement radius of 1.83 cm averaged over the suite's ten tasks — so a memorised place point is wrong by 14 cm against a target that moves by under 2, and 14 cm is enough to release a grocery over open table. **Only radial distance is measured**: the angles are chosen for legibility and carry no information, which is why no coordinate grid is drawn.

three commands — a sharper statement of layer 4 than we had, and one no cell we ran before could have detected.

8.1 Randomisation is a curve, not a point

Our ±4 cm radius was chosen to sit inside the controller's ±6 cm probe envelope — a perturbation calibrated to the system under test. Sweeping it on the same draws separates two things that single number confounded:

displacement	released head	Wilson 95%
none‡	4/10	[0.17, 0.69]
±2 cm	7/10	[0.40, 0.89]
±4 cm‡	4/10	[0.17, 0.69]
±6 cm (envelope edge)	2/10	[0.06, 0.51]
±8 cm (outside)	3/10	[0.11, 0.60]

Table 13: Sweeping the placement-randomisation radius on the same draws. Randomisation robustness is a curve, and at n=10 this curve is not even monotone — the undisplaced cell sits *below* ±2 cm, which cannot be a real benefit of displacement. Reporting any single radius as "passes randomisation" would quote a number this sample does not resolve, and we do not use this table to set any threshold elsewhere in the paper. ‡Per-trial outcomes for the ±2, ±6 and ±8 rows are in `results/pod_cells.json`; the two marked rows were measured in an earlier session and their trial records are not in the repository.

Read conservatively, because n=10 supports no more: clearly worse at ±6–±8 than at ±2, but every interval is ∼0.5 wide and the undisplaced cell sits *below* ±2 cm, which cannot be a real benefit of displacement. That non-monotonicity is the honest measure of what this n resolves. **No single radius should be quoted as "passes randomisation"**, and a benchmark asking for one is asking for a number that does not exist.

9 Certification

If a benchmark score cannot certify grounding, what can? This section reports three results about the certification problem itself, two of them negative.

9.1 Probes alone cannot certify

A substitution probe asks which input channels a head reads: swap one channel between two recorded ticks and measure how far the predicted grasp offset moves. It is annotation-free, cheap, and on-manifold — and that last property is fatal on its own.

One detail in Table 14 decides whether the probe means anything: it profiles the grasp *offset*, not the absolute point. The head emits $\text{eef}_{xy} + \Delta$, so substituting proprioception moves the absolute prediction by the entire arm displacement for *any* head, and profiling that quantity reports ∼24 cm of "proprioception attribution" even for a repaired one.

We made exactly this error when regenerating the measurement. It is the difference between a probe that answers "does this channel change where the head thinks the object is" and one that measures the parameterisation.

head	uv	proprio	box_emb	frame_emb
v2	0.75	2.85	6.04	6.91
v2.1	0.71	2.13	6.26	7.58
v5 (released)	0.97	1.93	3.43	6.57

Table 14: Substitution-probe attribution, in cm of movement of the predicted grasp *offset*, per input channel. The counterexample is the first two rows: v2 and v2.1 agree to within 0.73 cm on every channel and score 0.000 and 0.700 when deployed — these are heads v2 and v2.1, an earlier pair, and their 0.700 is not the released head's 35/50. A probe evaluated on teacher trajectories cannot, on its own, certify off-manifold behaviour.

Two earlier heads, v2 and v2.1, differ by at most 0.73 **cm on any channel and score** 0.000 **and** 0.700 **deployed.** The absolute framing flatters us: the largest gap is 34% in relative terms, on proprioception, the exact channel Layer 3 concerns. What we can defend is that no channel *ordering* distinguishes them. And the substitution is between a *single* pair of recorded ticks — the two most separated in *uv* out of 329 — with no resampling over pairs and therefore no interval on the 0.73 cm. It is an existence proof that two heads can be probe-identical and behaviourally opposite, not an estimate of how often that happens.

A probe evaluated on teacher trajectories is an on-manifold instrument and cannot, alone, certify off-manifold behaviour. Probe evidence must be paired with behavioural randomisation.

9.2 An instrument only ever shown firing is not yet an instrument

Our cross-instruction detection-agreement probe asks whether two objects' prompt chains select the *same box from the same frame*; a high same-box rate says the grounding stage carries no object identity. Every published number from it was a firing. Running it against every other object in the scene splits cleanly (Table 15):
The instrument registers difference when difference exists. The distances are close to bimodal — collisions at ∼0.004, separations at ∼0.82 — though not perfectly: milk and orange juice sit at 0.573 and 0.576, and they are excluded by the $n \geq 3$ gate rather than by the geometry.

The tolerance sweep needs stating in full, because we first reported only part of it. Over $\tau \in \{0.005, 0.01, 0.02, 0.05\}$ every verdict is unchanged. At $\tau = 0.1$, which the artifact also computed and our first write-up omitted, three of the separating contrasts move from 0.00 to 0.33. **Reporting the four tolerances at which nothing moves and not the one at which something does is the selective sensitivity reporting this paper is about**, so the full sweep is in `results/probe_positive_control.json` and the verdict should be read as holding for $\tau \leq 0.05$. The split is

counterpart	n	same-box	median dist.
cream cheese, butter, pudding	6	**0.00**	0.817–0.818
tomato sauce	4	1.00	0.00069
bbq sauce, salad dressing	5	0.80	0.0043
ketchup	5	0.60	0.0048

Table 15: The positive control our detection-agreement instrument had never been given. "Same-box" is the fraction of frames on which two objects' prompt chains select the identical box; a high rate means the grounding stage carries no object identity. Run against *every* counterpart rather than only the ones that fire, the instrument separates cleanly — and the split is interpretable: the chains that collide are the cans and bottles, the ones that separate are the boxes. Deployed binding is blind *within* a shape class, which is the regime LIBERO-Object's groceries occupy. Small n (only 4–6 of 60 frames have both chains detecting) makes these demonstrations of discrimination, not calibrated rates.

interpretable and sharpens the claim: the chains that collide are the cans and bottles, the ones that separate are the boxes. **Deployed binding is blind *within a shape class***, which is precisely the regime LIBERO-Object's groceries occupy.

Two limits. Only 4–6 of 60 frames had both chains detecting, so these are small-n demonstrations of discrimination, not calibrated rates. And the contrast we originally designated the positive control — the basket — is not evaluable at all: the deployed source-area filter rejects basket-sized boxes by construction, so it compares zero frames. Our first version of the script read same_rate = 0 off that empty comparison and printed "instrument discriminates". A verdict now requires $n \geq 3$.

9.3 Matching a rate is not reproducing a sample

We rebuilt the deployment stack on fresh hardware, pinning every version in §4 and verifying both load-bearing checkpoints by digest. The reference cell reproduces: 8/10 [0.49, 0.94] against a recorded 7/10, with 0.700 inside the interval.

Reproducing the *rate* is not reproducing the *experiment*. Holding seed, trial, weights and every package version fixed and changing only the OpenGL backend MuJoCo renders through (Table 16):
One disclosure before the reading. The `egl` cell carries `complete: false` and `planned_n: 0` in `results/pod_cells.json`: it was exploratory, not a planned cell, and our own harvester rule — report nothing that has not reached its planned n — should have excluded it. We keep it because it is the only such comparison we have and dropping it would bury the disclosure, but it does not meet the standard we set in Table 17.

The aggregate is identical and 40% **of the episodes are not** — but we can no longer attribute that to the backend *change*.

	`osmesa`	`egl`
success rate	8/10	8/10
per-trial outcomes	**4 of 10 disagree** (2 each way)	

Table 16: Changing only the OpenGL backend MuJoCo renders through — same seed, same trials, same weights, same package versions. The aggregate is identical and 40% of the individual episodes are not — though §9 withdraws the attribution of that difference to the backend change itself. Matching a published rate is therefore weak evidence of having reproduced an experiment, and any paired statistic computed across a renderer change is invalid, because pairing assumes the trials are the same trials.

MuJoCo's software and EGL renderers are documented as nondeterministic *within* a backend, so repeated renders of one state need not agree, and a 4/10 disagreement is consistent with that alone.

The control this needs is osmesa against osmesa at the full horizon. The one we have runs 40 steps and stops before the gripper closes — the contact phase where any such divergence would show.

We report the disagreement and withdraw the causal reading — the same error this paper's conclusion warns against. Thread count is a remark rather than a result, for the reason just given: three `osmesa` runs at 1, 4 and 128 threads agree on all eight logged fields while the wall clock falls from 220 s to 32 s. We state the limit plainly — that comparison ran to 40 steps, a prefix in which the gripper never closes, so it does not probe the contact-rich part of an episode where the renderer difference showed up (`results/thread_determinism.json`). Software and GPU rasterisation do not produce identical pixels, the detector is not invariant to that difference, and in this stack the detector is the only encoder.

Two consequences. Matching a published rate is weak evidence of having reproduced an experiment: the marginal can be right while the sample is different. And *any paired statistic computed across a renderer change is invalid*, because pairing assumes the trials are the same trials.

We must therefore report our own compliance honestly. Every paired test in §6 is within one build, because those cells were produced in one sweep. We cannot say the same for the whole paper: the confound test of §11.1 ran on a different machine entirely — macOS, MuJoCo 3.3.0, robosuite 1.4.0, Python 3.13, recorded in its artifact — and no per-cell build ledger exists for the older cells. Given what this subsection demonstrates, that is a gap in our own protocol and not a footnote. `MUJOCO_GL` belongs in any reported stack; it was not in ours, or in any LIBERO evaluation we have read.

10 Transfer: a negative result

The instruments above are offered as portable, so we tried them on a policy we did not build: the public `openvla-7b-finetuned-libero-object` checkpoint [6], through the same harness, trial-to-state map and success predicate.

The baseline returned 0/1 on a checkpoint published near 0.9. We treated that as a harness defect and found four: image orientation, the gripper convention (their model emits $[0, 1]$ with 1=close, robosuite wants $[-1, +1]$ with -1=close — without the conversion the jaw never closes), success extraction (the environment's `done` flag also fires on horizon), and a missing signal shield. We then swept four image orientations against the 0.9 centre crop their evaluation applies.

The checkpoint still does not approach the target in any configuration we tried. **We therefore report no OpenVLA numbers at all** — not success rates and not distance statistics — because the diagnostic runs were exploratory and were never written to a versioned artifact, and an unvalidated baseline would make our instruments look powerful for the wrong reason.

All four defects biased in the same direction — toward the checkpoint looking worse — which is precisely how an external-validation result gets fabricated without anyone intending it. We cannot separate a remaining defect in our harness from a checkpoint that does not transfer to a differently-built LIBERO, and shortfalls in reproducing this checkpoint's published LIBERO-Object number are reported by others on the project's issue tracker, so we treat the gap as unexplained rather than as a finding of ours. The harness ships (`eval/openvla_eval.py`). Until someone with the matching build runs it, **the portability of these instruments is unestablished**.

11 Negative results and registries

A paper that only reports what worked cannot be checked. This section is the ledger: what we predicted before running, what failed, which instruments we retired, and the nine ways our own results fooled us.

Pre-registration. Fourteen predictions are on record with their outcomes; seven failed. Two failures matter here: the coverage head's swap cell did not hold (p=0.031), and the released head was not at floor outside the controller's envelope (3/10).

The blind arm carries *no* registered prediction — it was implemented in one compute window and run in the next — and neither does the attribution analysis that overturned it (§6), which was formed after the registered hypothesis failed. Neither is reconstructed after the fact.

Retired cells. Two cells from earlier rounds are retired rather than silently dropped. An n=50, ±4 cm displace-

ment cell scoring 0.520 is superseded by the per-radius curve of Table 13, which resolves the shape of the response where a single radius could not. And a free-regression variant scoring 0/56 is withdrawn: it was measured on a configuration this paper no longer describes, and we have no per-trial record for it.

Retired instruments. Quantifying deployed role binding was attempted six times and abandoned six times, each at a calibration gate written down before the instrument was trusted. Deployed binding accuracy is reported as **unmeasured**. The six: a projected-origin ground truth (the object that succeeds 35/50 scored 0.111), its sign-corrected successor (the in-frame filter deletes the close-up target by construction), three text-tower discrimination probes (each failed to detect the working object's own phrase), and a box-spread metric (confounded by camera motion, and inverted).

11.1 A confound we had missed

Our own analysis of a blocked object was confounded: because the suite's two placement clusters align with which objects we happened to train on, "blocked" and "22 cm from the training position" were perfectly correlated in our design. Tested directly over all ten objects, position predicts the effect weakly (Spearman +0.19, p=0.60) and *teleporting each object 22 cm shows no directional effect* (6/10, exact sign test p=1.0, the least informative value the test returns), while detector groundability predicts it (-0.71, p=0.027). The mechanism is appearance, not position — position is a nuisance term with mean $|\Delta| = 0.0066$, which is not zero either.

This does not contradict §3, and the distinction is worth stating. Those sections answer different questions. Placement is what the suite fails to randomise and therefore what a policy *may exploit*: that is a property of the benchmark, established from its shipped files, and it is what layers 1 and 3 were caught doing. Appearance is what limits *transfer to a new object* once that shortcut is repaired: a property of the frozen detector, and why the released head works on one object and not on nine (§6).

A benchmark can be lookup-admissible and a policy can still fail for an unrelated reason. Both are true here, neither weakens the other, and what *would* have been a contradiction — position explaining both — is what the confound test ruled out.

We also report the caveats on this cell rather than only its conclusion. Both nulls are n=10, which is the size we decline to accept elsewhere (§8); the groundability predictor and the outcome are two statistics of the same detector forward pass, so their correlation is closer to a self-consistency check than to an independent prediction; and $p = 0.027$ across three tests does not survive a Bonferroni correction. The confound is *disconfirmed as a sufficient explanation*, which is what the design can support, and not more.

11.2 Nine ways a result fooled us, and the one thing they share

Recorded because the failure modes generalise and *none* was caught by a conventional check. In every one of them the instrument was correct (Figure 12; repairs in Table 17).

That is the pattern (Figure 12), and it took nine instances to see it. Not one of these was a bug in a measurement. Each was a decision about *which slice got written down*: which trials had finished, which axes were compared, which tolerances were swept, which tasks were counted, which of two estimates was quoted — and, in the eighth, which parts of a document a retraction reached. **A verifier that recomputes every number in a paper would have caught none of them**, and ours did not — it recomputed the 2D distance we chose to compute, and passed.

They also share a direction. All nine made the result look better. A team that only checks its arithmetic is protected against none of this; what catches it is being asked, by someone else, why a particular slice was the one reported.

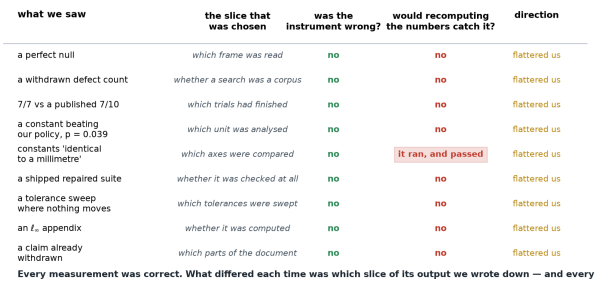
12 Discussion

What a score certifies. On a suite whose placement radius sits below the embodiment's tolerance, a success rate certifies that the policy can execute a manipulation *given* a correct target pose — and nothing whatever about how it obtained one. That is a statement about what the score *could* distinguish, and it is exactly as strong as A1.

The narrow statement is deliberate: we already overreached once. Our own cells do not demonstrate that any policy exploited the degeneracy; §6 shows they cannot. What the radius buys is a check that runs before training, and a prediction — that a suite with small radii admits a cheap solution — which Jiang et al. [5] confirm on LIBERO with a trained probe where we did not with a constant.

Shortcuts live in pipelines, not only in models. Two of our four layers are outside the network: a scripted expert's calibration constant and our own checkpoint-selection loop. No amount of perturbing a trained model from the outside would have found either. This is an argument for glass-box artifacts in benchmark-validity work, and it is the main reason our stack is small.

Certification is a system property. Probes cannot certify alone: two heads whose channel *ordering* a substitution probe cannot separate score 0.000 and 0.700 deployed. Behaviour must be paired with randomisation.



what we saw	the slice that was chosen	was the instrument wrong?	would recomputing the numbers catch it?	direction
a perfect null	*which frame was read*	no	no	flattered us
a withdrawn defect count	*whether a search was a corpus*	no	no	flattered us
7/7 vs a published 7/10	*which trials had finished*	no	no	flattered us
a constant beating our policy, p = 0.039	*which unit was analysed*	no	no	flattered us
constants 'identical to a millimetre'	*which axes were compared*	no	it ran, and passed	flattered us
a shipped repaired suite	*whether it was checked at all*	no	no	flattered us
a tolerance sweep where nothing moves	*which tolerances were swept*	no	no	flattered us
an ℓ_∞ appendix	*whether it was computed*	no	no	flattered us
a claim already withdrawn	*which parts of the document*	no	no	flattered us

Every measurement was correct. What differed each time was which slice of its output we wrote down — and every error ran the same way.

The fifth row is the one we can state as fact rather than counterfactual: our verifier recomputes every number in the load-bearing sections from raw outcomes, it passed 76 checks, and the claim it certified was false — because it recomputed the two-dimensional distance we had chosen to compute.

Figure 12: **Nine errors, and why checking the arithmetic caught none of them.** One row per error, with the repair in Table 17. The three right-hand columns are the finding: the instrument was never wrong, a checker that recomputes every number would not have caught it, and every error ran in the direction that flattered us. The fifth row is fact rather than counterfactual — our verifier recomputes every number in the load-bearing sections from raw per-trial outcomes, it passed 76 checks, and the claim it certified was false, because it recomputed the two-dimensional distance we had chosen to compute. Generated by `paper/render_fig14_errors.py`.

The renderer result points the same way but we state it at the strength it now has. Holding weights, seeds and package versions fixed and changing only the OpenGL backend leaves the rate identical and 40% of the individual episodes different — and §9 withdraws the attribution of that to the backend *change*, because MuJoCo is documented as nondeterministic within a backend and our same-backend control runs only to 40 pre-contact steps. What survives is the weaker and still useful claim: **matching a published rate is not evidence of having reproduced a sample**, so a paired statistic must be computed within one build.

Limitations.

- One simulator, one benchmark suite, one robot, wrist camera only.
- Every confirmatory cell is one task and one object at $n \leq 30$, including the blind arm (n=10). The *radii* of §3 are computed from files and carry no such limit; the *verdict* does, because it needs δ.
- One 10-trial cell (the released head on task 0, seed 20) appears in five reported results: the suite total, the blind comparator, the renderer table's `osmesa` arm, the instruction-swap baseline, and the reproduction check. It is one sample, not five.
- The held-out band is partially burned; the disclosure is in §4 and the bound in §8.
- The on-manifold limit of substitution probes is a limit on our own instruments, not only on others'.
- Transfer to an external policy is **unestablished** (§10).
- R is defined for suites shipping enumerable initial states, and would need restating for procedurally generated ones.

12.1 Running this on a benchmark that is not LIBERO

Both measurements need the same two things (Table 18), and neither needs a model: a statement of where each task's objects *may* start, and the states the benchmark actually *ships*. LIBERO supplies the first as BDDL `:ranges` and the second as `.pruned_init` arrays. Any benchmark with a generator and a released evaluation set has both, in some form.

We flag one thing a porter should not assume. Our region parser reads the *assignment* of object to region from the task's own initialisation block, because matching regions by name silently picked the plate's region for the bowl and reported a task using 152% of its declared area. Whatever the target format, the object-to-region binding must be read, not inferred.

12.2 Recommendation to benchmark authors

Both quantities are cheap to compute and cheap to report. Table 19 is what we would ask of a suite.

13 Conclusion

A manipulation benchmark score is not a measurement of capability until the benchmark's admissibility has been measured. The measurement is cheap, it runs before any model is trained, and on LIBERO it says something specific: the one suite whose tasks are individuated by object identity is also the one that pins that object, on all ten tasks, to a radius smaller than the smaller of the two tolerances we measure for our own arm — the larger one dissolves the contrast, and we report both.

One behavioural claim on top of that does stand, and we tested it the way we would want someone else to. Sam-

pling the region a task file declares, and changing nothing else, costs a lookup policy half its score ($25/30 \to 10/30$, $p = 0.0007$): the defect is exploitable, not cosmetic. That is the only causal statement in this paper, it is one task at n=30, and its own obvious confound gets an arm of its own rather than a paragraph.

The larger claim — that a constant matches a trained policy — does not stand, and the way it fell is the more useful half of this paper. We ran the lookup policy, saw it outscore our trained one, tested it at the wrong unit of analysis, and reported a significance that a task-level test does not support. The deeper error was attributional: three tasks handed a constant 0.469 mm from a fourth's, in all three axes, score 0/10 where it scores 10/10 — so the successes were never evidence about lookup at all. Both errors were found by adversarial review of a manuscript we already believed, using data we had already published, and seven more like them followed — nine in all.

What survives is the procedure rather than the artifact. Measure the benchmark first, at the unit the design randomises over. Check that the variable you manipulated is the one that moved. And report the reading that did not survive being checked, which in this revision is repeatedly our own.

One lesson generalises past this paper, and it took nine instances before we saw it. Every error we made was a correct instrument reporting a chosen slice: which trials had finished, which axes were compared, which tolerances were swept, which of two estimates was quoted. **Recomputing every number in a paper catches none of that** — our verifier recomputed the two-dimensional distance we had chosen to compute, and passed. What catches it is being asked, by someone who did not choose the slice, why that slice.

A Recovering which column owns which object

LIBERO ships each task's initial states as a MuJoCo flattened state $[\,t \mid q \mid \dot{q}\,]$; every shipped $\dot{q}$ is zero, so the columns that vary across states are exactly the position degrees of freedom the suite randomises.

Recovering *which* object owns which column requires care. The fixed layout $[\,t \mid 9 \mid N \times 7 \mid \dot{q}\,]$ that works for LIBERO-Object predicts a width of $19 + 13N$ and fails for every suite containing an articulated fixture — LIBERO-Spatial ships width 92 against 5 declared objects. We therefore run two passes: the fixed-layout one, and one that compiles each task's model and reads `jnt_qposadr` with the joint names, mapping every column onto a named body. **The two agree wherever both apply**, and only the second is used for suites the first cannot parse. Artifacts: `results/suite_forensics_columns.json` (digests), `results/suite_forensics_joints.json` (per-object positions).

B The norm, recomputed

Definition 1 uses the Euclidean norm; A1 is naturally stated in ℓ_∞, because LIBERO randomises placement in an axis-aligned box. A previous version of this appendix asserted, without computing it, that the switch changed one LIBERO-Spatial task and left the comparison intact. That was wrong, and a referee caught it. Computed (`results/admissibility.json`), at $\delta = 1.4$ cm:

suite	ℓ_2	ℓ_∞
Object	10/10	10/10
Spatial	0/10	0/10
Goal	2/10	**3/10**
Long	0/10	0/10

Under ℓ_∞ the "both are fixtures" clause fails. The third LIBERO-Goal task to cross is `put_the_bowl_on_the_stove`, whose bowl moves. The ℓ_∞ separating window is $[0.97, 1.38)$ cm, which *excludes* the $\delta = 1.41$ cm we use. So in the norm we ourselves call natural for A1, the headline does not hold. We report the Euclidean version because it is the one our δ was measured in, and we flag that this is a choice the result is sensitive to.

C Sensitivity of the verdict to δ

This appendix is where the paper's headline is most vulnerable, so Table 20 gives the curve at fine granularity rather than at the points that suit us. A previous version of this table stepped from 1.49 directly to 2.0 cm, which happens to skip the interval containing our *own* second tolerance estimate. That was not deliberate and it was not defensible.

Three reasons to distrust δ, stated together. First, our two estimates disagree by a factor of 1.4 and the headline holds only under the smaller. Second, even within task 0 the classes overlap — the largest success sits at 1.45 cm and the smallest failure at 1.41 cm — so there is no clean threshold at n=10, only a rank association.

Third, and most serious: both estimates are measured on the *container* during the place phase, and Table 6 applies the result to the *grocery* during the grasp phase. A parallel-jaw grasp tolerance on a 6 cm can and a release tolerance into an open basket are not the same physical quantity, and we have not measured the first at all.

The experiment that would fix all three is the same one: sweep a deliberate displacement of the grasp constant, per object, and read the radius off the curve. It is implemented (`--goal-anchor-jitter-cm`) and unrun.

References

[1] Pim de Haan, Dinesh Jayaraman, and Sergey Levine. Causal confusion in imitation learning. In *NeurIPS*, 2019.

[2] Senyu Fei et al. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.

[3] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020.

[4] Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In *Proceedings of NAACL-HLT*, 2018.

[5] Tianchong Jiang, Xiangshan Tan, Samuel Wheeler, Luzhe Sun, Tewodros W. Ayalew, and Matthew Walter. What are we actually benchmarking in robot manipulation? *arXiv preprint arXiv:2606.04233*, 2026.

[6] Moo Jin Kim et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[7] Ajay Mandlekar et al. What matters in learning from offline human demonstrations for robot manipulation. In *CoRL*, 2021.

[8] R. Thomas McCoy, Ellie Pavlick, and Tal Linzen. Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. In *Proceedings of the Association for Computational Linguistics (ACL)*, 2019.

[9] Wilbert Pumacay, Ishika Singh, Jiafei Duan, Ranjay Krishna, Jesse Thomason, and Dieter Fox. THE COLOSSEUM: A benchmark for evaluating generalization for robotic manipulation. In *Robotics: Science and Systems (RSS)*, 2024.

[10] Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do imagenet classifiers generalize to imagenet? In *International Conference on Machine Learning (ICML)*, 2019.

[11] Jesse Thomason, Daniel Gordon, and Yonatan Bisk. Shifting the baseline: Single modality performance on visual navigation and QA. In *NAACL*, 2019.

[12] Antonio Torralba and Alexei A. Efros. Unbiased look at dataset bias. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011.

[13] Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.

what we saw	what it was	the general lesson
a perfect null: teleport gaps identical to control for all ten objects, to four decimals	a cached observation — the renderer returned the *pre*-teleport frame	implausible cleanliness is a symptom. The script now compares frames across the intervention and *raises* rather than emitting
a defect count withdrawn as unsupported	the appendix we could not find existed	"I could not find it" is a claim about a search, not about a corpus
7/7 against a published 7/10; we began writing up a failure to reproduce	a censored sample. The complete cell is **8/10**	successes end episodes early and failures run to the horizon, so any interim harvest is biased toward successes. **Verifying the instrument does not verify the sampling**
a constant outscoring our trained policy 16/100 to 8/100, $p = 0.039$	ten replicates of one task counted as ten trials — and a constant that did not cause the successes at all	we checked the numbers and not the *unit*, nor whether the variable we manipulated was the one that moved
constants "identical to within a millimetre" with opposite outcomes	compared in the table plane; in 3D the groups differ by 40 mm on z	the arm is handed a 3-vector. Our verifier reproduced the error because it dropped z too
a repaired evaluation suite, shipped	free sampling of the declared box would have placed objects in contact on 5 of 10 tasks	we validated it only after shipping. The assumed object radius (5 cm) was wrong; measured from the assets it is 2.2–3.7
a tolerance sweep in which nothing moves	the one tolerance at which three verdicts *do* move was computed and not reported	selective sensitivity reporting, in the section arguing against it
an appendix reporting the ℓ_∞ variant	asserted, never computed; recomputed it says the opposite	a sentence with no artifact behind it reads exactly like one that has
a claim we had already withdrawn, still standing	withdrawn in the section that made it, and left intact in the discussion, the contributions list and two captions that cite it	**retracting a claim where it was made does not retract it where it is summarised**, and summaries are the last place anyone re-reads

Table 17: Nine self-inflicted errors and their repairs. Every one was found by someone checking a choice rather than a computation, and every one ran in the direction that flattered us. The third is the one we would most expect others to hit: every check we ran had passed, and the defect was in *when* we looked. The harvester now reports each cell against its planned n and refuses to report an incomplete one.

you need	to compute	which tells you
shipped initial states, per task	the placement radius R	whether one constant per task reproduces every shipped position
the declared sampling region	declared-versus-shipped	whether the released states exercise the randomisation the benchmark specifies
+ a tolerance δ for the embodiment	the admissibility verdict	whether that matters for a given robot — and this is the part that needs a policy

Table 18: What the two diagnostics require. The first two rows are arithmetic on shipped files and are what we would ask a benchmark to publish; the third needs an embodiment and is where our own result is weakest. The scripts are `scripts/admissibility.py` and `scripts/declared_vs_shipped.py`; both read a per-task table of object positions and are ∼150 lines. Porting them is a matter of writing the reader for another benchmark's state format, not of reimplementing the measurement.

practice	cost	why
publish R per object per task	one script, no training	tells a reader what a score can certify
set a floor on R	re-sample the init files	a suite individuated by identity must randomise placement, or identity is never load-bearing
ship a tier of radii	k extra eval configs	where a policy degrades depends on displacement relative to the controller's envelope (Table 13); one radius is a calibration, not a result

Table 19: Three practices, in increasing order of what they cost a benchmark's authors. LIBERO-Spatial already satisfies the second by necessity — all ten of its targets are the same bowl — and LIBERO-Object could satisfy it by choice.

δ (cm)	Object	elsewhere	
1.17	9/10	2/30	
1.40	**10/10**	**2/30**	← task-0 estimate
1.49	10/10	2/30	
1.60	10/10	3/30	
1.70	10/10	7/30	
1.80	10/10	9/30	
1.91	10/10	**13/30**	← task-3 estimate
2.00	10/10	17/30	
2.50	10/10	19/30	

Table 20: Tasks whose identity-bearing object one constant serves, against the tolerance. **The separation is real at** 1.4 **cm and gone by** 1.9 **cm**, and those two numbers are our own two measurements of the same assumption (§6): task 0 fails at basket displacements from 1.41 cm, task 3 tolerates 1.91 cm. We report the headline at the smaller one, which is the value that favours us. At the larger one LIBERO-Object is still 10/10 but so are thirteen tasks elsewhere, and the contrast is no longer a separation. Artifact: `results/admissibility.json`.